CS5800 · Spring 2026

Apartment Commute Optimizer

A data-driven framework for evaluating apartment convenience via public transit

Xujing Hui · Fanchao Yu · Jiaxin Liu

5
Apartments

25
Destinations

36
Transit Nodes

4
Categories

github.com/Xujing-Hui/apartment-optimizer

The challenge

OVERVIEW

What are we solving?—The Apartment-Hunting Problem

Students in the Bay Area choose apartments based on rent and gut feeling.

But "close to campus" ignores the full picture: gym 4×/week, groceries 2×/week, coffee 3×/week — these add up.

Over a 40-week academic year, picking the wrong apartment can cost **48 extra hours of commute.**

NEU Campus
3 visits/week (via Starbucks)

24 Hour Fitness
4 visits/week

Trader Joe's
2 visits/week

Costco
1 visit/week

→ **We need an algorithm to rank apartments by total weekly commuting cost.**

The Problem Statement

OVERVIEW

Among 5 candidate apartments in the San Jose metro area, which one minimizes total weekly transit commute time to school, gym, groceries, and daily coffee — subject to rent and transit-access constraints?

Inputs

5 candidate apartments across 4 cities

25 destinations (Starbucks, Costco, TJ, Gym, NEU)

36 transit stations as intermediate nodes

Output

Apartments ranked by weekly commute time

Best store picked per category by the algorithm

Shortest path from each apartment to each destination

Constraints

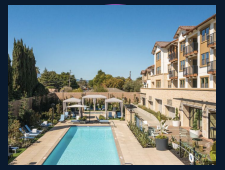
Monthly rent ≤ $4,000

Walk to nearest station ≤ 15 min

The 5 Apartments

DATA — APARTMENTS

All within budget ($4,000/mo) and ≤ 15 min walk to transit

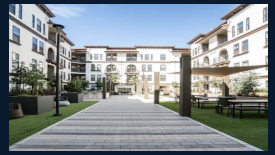


Murphy Station

$3,817

Sunnyvale

🚌 Sunnyvale Caltrain
8 min



Villas on the Boulevard

$3,465

Santa Clara

🚌 VTA 522 Rapid
4 min

Cannery Park by Windsor

$3,359

Downtown SJ

🚌 SJ Diridon Caltrain Station
5 min

The Verdant Apartments

$3,292

North San Jose

🚌 VTA 72 + Rapid 500
5 min

The Harlowe

$3,300

Milpitas

🚌 VTA 20 / Rapid 500
7 min

Apartments span 4 cities to ensure Dijkstra produces meaningfully different paths for each.

Source: Zillow, Apartments.com — April 2026. Coordinates verified via Google Maps.

25 Destinations + 36 Transit Nodes

DATA — DESTINATIONS

NEU Campus

1 location

3×/week

Fixed endpoint of chain trip

4 N 2nd St, San Jose

Starbucks

6 locations

3×/week

Always visited before NEU
— the chain trip waypoint

S1 Sunnyvale
S2–S3 Santa Clara
S4 Zanker
S5 Downtown
S6 Milpitas

Costco

6 locations

1×/week

Lowest visit weight

C1 Sunnyvale
C2 Santa Clara
C3 SJ Auto
C4 Mtn View
C5–C6 SJ

Trader Joe's

6 locations

2×/week

Algorithm picks nearest
of 6 Bay Area locations

T1 Sunnyvale
T2 SJ Coleman
T3 Milpitas
T4 Mtn View
T5–T6

24 Hour Fitness

6 locations

4×/week

Highest visit frequency
— biggest weight in score

G1 Sunnyvale
G2 Santa Clara
G3–G4 SJ
G5 Milpitas
G6 SJ

Transit Network:

36 nodes: Caltrain stations + VTA Rapid 500/522 + VTA local bus (20/22/57/72/73) + Milpitas BART
Total graph: 66 nodes, ~94 edges, all weights in minutes

Transit Graph Structure

ARCHITECTURE — GRAPH

3 JSON files form a unified undirected weighted graph

apartments_json.json

5 nodes — walk_min_to_station

↓

stations_json_v2.json

36 nodes — Caltrain · Bus · BART · Hub

↓

destinations_json.json

25 nodes — walk_min_from_station

merged into →

Unified Weighted Graph G = (V, E)

66 Total Nodes	**~95** Edges (undirected)	**5** Apartments (source)
25 Destinations (sink)	**36** Transit nodes	**10** Walk fallback edges

Path pattern: Apt → Station → ... → Station → Dest

- **walk** — pedestrian transfer between co-located nodes
- **caltrain** — Caltrain schedule time
- **bus** — VTA in-vehicle time
- **fallback** — haversine walk ≤ 30 min direct edge

The Algorithm

Dijkstra's shortest path + walk fallback + category minimum + weekly weighting

1 Build Graph

Connect apartments, stations, and destinations via edges weighted by travel_min. All edges undirected.

2 Walk Fallback

For any (apt, dest) pair and any (Starbucks, NEU) pair where haversine walk ≤ 30 min, add a direct edge. Activates 10 pairs in our dataset

3 Run Dijkstra

One run per apartment ⟶ shortest path to all 66 nodes simultaneously. Precompute from NEU once for chain trip efficiency.

4 Chain Trip

Starbucks + NEU: find S* = argmin [d(Apt⟶S) + d(S⟶NEU)]. S* is not always the nearest Starbucks.

5 Weighted Score

Multiply each category cost by weekly visits. Sum = total weekly commute minutes. Lower = better.

Objective Function

```
score(apt)  =
3 × min_s[ d(Apt→S) + d(S→NEU) ]
  + 1 × min( d → Costco )
  + 2 × min( d → Trader Joe's )
  + 4 × min( d → 24HF Gym )
unit: minutes / week
```

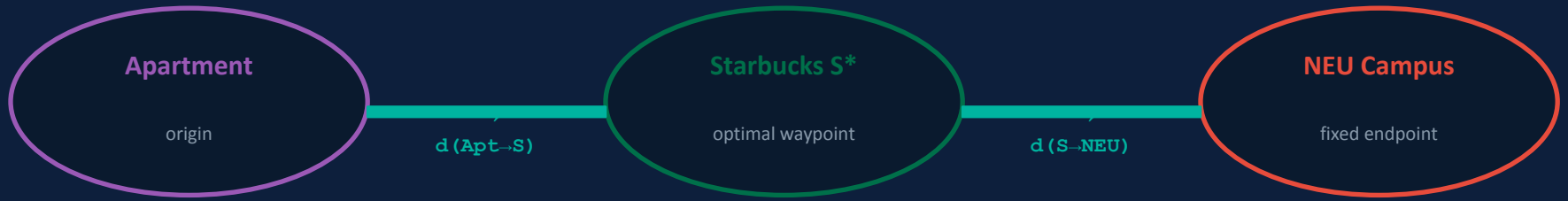
Visit weights: ×3 School ×3 Starbucks ×1 Costco ×2 TJ ×4 Gym

Gym (×4) = highest weight — 4 visits/week dominates total score

The Chain Trip: Starbucks ⟶ NEU

ALGORITHM — CHAIN TRIP

Every school visit includes a Starbucks stop — order matters: Starbucks first, NEU second



Why optimize S*?

The nearest Starbucks to the apartment is not always optimal. S5 (Downtown SJ) is closest to NEU, only 286m away, a 4.7-minute walk, so an apartment nearly equidistant to several Starbucks locations may prefer S5 — saving time on the longer S⟶NEU leg. The algorithm evaluates all 6 candidates and picks the global minimum of the two-leg path.

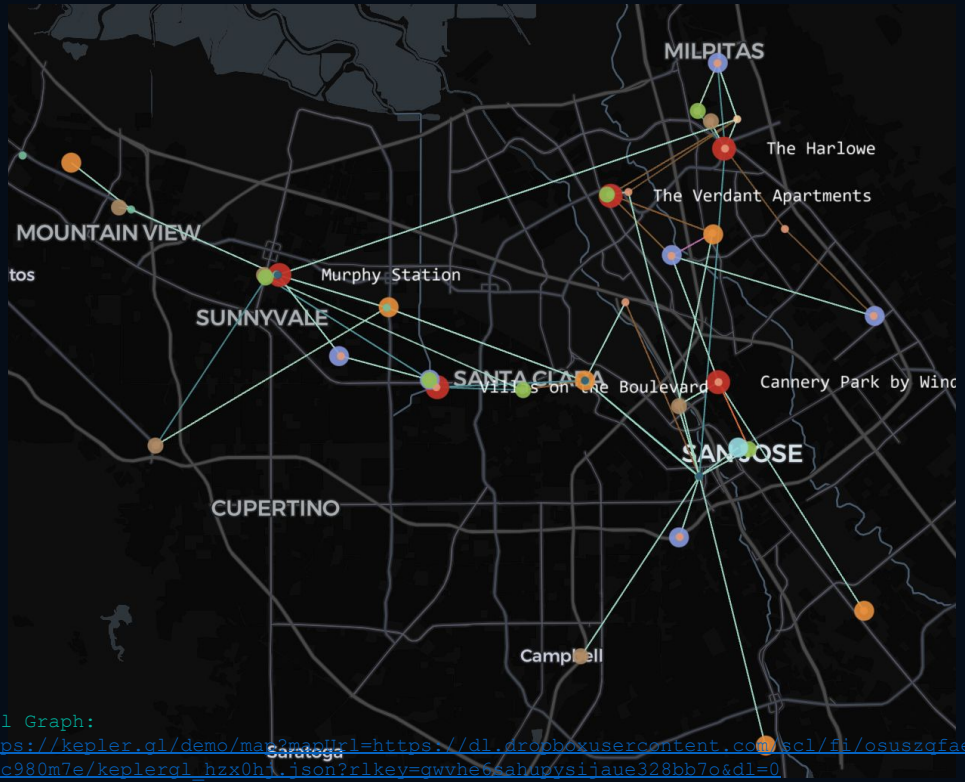
6 Starbucks Candidates

```
S1  Sunnyvale (W El Camino Real)
S2  Santa Clara (3605 ECR)
S3  Santa Clara (495 ECR / SCU)
S4  San Jose — Zanker Rd
S5  Downtown SJ (181 E Santa Clara)
S6  Milpitas (127 Ranch Dr)
```

Transit Network on Kepler.gl

VISUALIZATION

Interactive map — 10-point layers + 5 per-apartment path layers



Full Graph:
https://kepler.gl/demo/map?mapUrl=https://dl.dropboxusercontent.com/scl/fi/osuszgfaepo619c980m7e/keplergl_hzx0hj.json?rlkey=gwvhe66ahupysijaue328bb7o&dl=0

Demo Graph:
https://kepler.gl/demo/map?mapUrl=https://dl.dropboxusercontent.com/scl/fi/sqhhk2bjfh9v2a27ig2gx/keplergl_iyd6e3d.json?rlkey=ykf7hamjafzd680s03rjgxh3r&dl=0

Point Layers (10)

- Each destination category = 1 layer
- Apartments, NEU, Starbucks, Costco
- Trader Joe's, 24HF, Caltrain, Bus, BART

Path Layers (5)

- One GeoJSON per apartment
- Each segment color-coded by mode
- walk / Caltrain / VTA bus

Kepler Filtering

- Filter by apartment_id
- Isolate one apartment's route network
- Toggle layers to compare coverage

Scoring Results

RESULTS

Lower score = less total weekly transit time = more convenient apartment

Rank	Apartment	Area	Rent	Chain×3	Costco×1	TJ×2	Gym×4	Total (min/wk)
#1	Villas on the Boulevard	Santa Clara	$3,465	134.1	29	39.2	18.4	220.7
#2	Cannery Park by Windsor	Downtown SJ	$3,359	71.1	29	36	88	224.1
#3	Murphy Station	Sunnyvale	$3,817	125.1	18	13.4	80.4	236.9
#4	The Verdant Apartments	North SJ	$3,292	110.1	23	67.4	72	272.5
#5	The Harlowe	Milpitas	$3,300	149.1	45	26.4	72	292.5

★ Walk fallback edges (≤30 min) used: Villas↔S2 4.6min, Villas↔G2 4.6min, Murphy↔S1 6.1min, Murphy↔T1 6.7min, Verdant↔S4 1.7min, Cannery↔NEU_SV 29.9min, Cannery↔T2 20.1min, Harlowe↔T3 13.2min, Harlowe↔S6 20.1min, S5↔NEU_SV 4.7min

min/week

Compare with GenAI

Dimension	GenAI Approach	Our Algorithm
Routing model	Straight-line or Google Maps API	Dijkstra on 66-node multi-modal graph
Store selection	Human pre-selects nearest store	Algorithm discovers optimal via graph search
School commute	NEU scored independently	Chain trip: Apt → Starbucks → NEU
Walk handling	Ignored or 15-min blanket assumption	Haversine fallback edges, cap = 30 min
NEU computation	Repeated per apartment (30× waste)	Precomputed once; reused via undirected property
Transit network	Single-mode or approximate	Caltrain + VTA Rapid/Local bus + BART

Apartment	v1 Score	v2 Score	Change	Driver
Villas on the Boulevard	269	220.7	−48.3	Walk-fallback: gym 13→4.6 min, SB 12→4.7 min
Murphy Station	302	236.9	−65.1	Walk-fallback: TJ 20→6.7 min, SB 19→4.7 min
The Verdant	330	272.5	−57.5	Walk-fallback: SB S4 = 1.7 min
Cannery Park	309	224.1	−84.9	Chain walk-fallback: S5↔NEU 4.7 min walk
The Harlowe	346	292.5	−53.5	S5 now best Starbucks for Harlowe too

Time and Space Complexity

Component	Time Complexity	Space Complexity	Notes
Build graph (Steps 1–3)	O(V + E)	O(V + E)	One pass through neighbor arrays
Walk-fallback check (Step 4)	O(A × D)	O(1)	A=5 apts, D=25 dests = 125 checks
NEU precompute (1 Dijkstra)	O((V+E) log V)	O(V)	Runs once outside apartment loop
Dijkstra per apartment	O((V+E) log V)	O(V)	5 runs total, V=66, E≈94
Full pipeline (A apartments)	O(A · (V+E) log V)	O(V+E)	Dominated by Dijkstra runs
Bellman-Ford (verification)	O(V × E)	O(V)	Used in testing only
Category scoring	O(A × 6)	O(1)	Linear scan over 6 candidates each
Sorting	O(A log A)	O(A)	A=5, negligible

Key Takeaways

Visit frequency weights are lifestyle parameters

- Gym ×4 is the highest-weighted category and the largest source of score variance across apartments: 18.4 to 88.0 min/week.
- Villas wins because its gym contribution (4 × 4.6 min = 18.4 min/week) is the lowest of any apartment

Problem formulation matters more than algorithm choice

- Dijkstra is textbook. The real contributions were 3 modeling decisions: chain trips, walk-fallback edges, and letting the algorithm select the optimal store instead of pre-assigning one.

Weaknesses & Limitations

- In-vehicle time only, waiting time excluded.
- Walk-fallback uses Haversine × 1.3 detour factor. Actual pedestrian routes may differ.

Avenues for Future Research

- Real-time headway modeling
- Multi-stop chain generalization, eg: Apt ⟶ Stop1 ⟶ Stop2 ⟶ … ⟶ Dest
- Real estate industry pricing model or regional demographic preference research
- Further enhance scalability

Framework is extensible: add new destinations, adjust weights, or model different user routines — the graph and algorithm scale naturally.